

Dynamic Meta-Factory Automaton (DMFA)

A Universal Growth Function — Not a Number, But a Process

**Transcending Fixed Bounds Through Structural
Self-Parameterization**

Department of Theoretical Mathematics and Computation Theory

August 2026

Abstract

We present the **Dynamic Meta-Factory Automaton (DMFA)**, a deterministic algorithm that functions as a **universal growth operator** rather than a static constant. Unlike classical large-number constructions (Graham's Number, $TREE(3)$, $SSCG(3)$, Rayo's Number), which are fixed numbers regardless of their magnitude, DMFA is a **computable function** that accepts any natural number as input and produces a strictly larger output through structural self-parameterization.

The Core Mechanism: The input seed M is raised to itself (M^M), and this value is decoded into operational parameters — the operand v , the hyper-operational level op , the iteration count c , and (exclusively in the first factory) the pipeline width F . Each factory applies a hyperoperation and raises the result to its own power (R^R), producing the input for the next factory. After F factories, a grand reset compacts the final output into a new seed for the next cycle.

We prove that for **any** computable seed $M \geq 2$ and **any** finite number of cycles $k \geq 1$:

$$\boxed{\text{DMFA}(M, k) > M} \tag{1}$$

Furthermore, the sequence $\{\text{DMFA}(M, k)\}_{k=1}^{\infty}$ is strictly monotonically increasing and unbounded. This establishes DMFA as a **process of unbounded growth**.

1 Introduction: The Paradigm Shift from Numbers to Processes

The study of extremely large numbers has long fascinated mathematicians and computer scientists. Classical constructions such as Graham's Number [1], $TREE(3)$ [2], $SSCG(3)$, and Rayo's Number [4] establish fixed reference points within the Fast-Growing Hierarchy (FGH). These numbers, though incomprehensibly large, share a fundamental limitation: they are **static constants**.

The Fundamental Problem: A fixed number, regardless of its magnitude, is bounded by itself. $TREE(3)$ does not grow. It is simply large. Once constructed, it remains unchanged forever. Graham's Number, Rayo's Number — all are frozen at their respective magnitudes.

This paper introduces a **qualitatively different approach**: rather than constructing a bigger number, we construct a **process that makes any number bigger**. The Dynamic Meta-Factory Automaton (DMFA) is not a number — it is a **universal growth function**.

1.1 The Core Mechanism: How M^M Becomes Parameters

The essence of DMFA lies in its **decoding mechanism**. Given any input seed M :

1. **Self-Exponentiation:** Compute $X = M^M$
2. **Structural Decoding:** Extract operational parameters directly from the value of X
3. **Factory Execution:** Apply hyperoperations using these self-derived parameters
4. **Re-encode (Self-Compaction):** Raise the result to its own power, creating the next input

This creates a closed loop where the output of each step encodes the parameters for the next step — a true **self-parameterizing system**.

Key Insight: The number $X = M^M$ is not merely a value to be operated upon. It is a **self-encoding data structure** that contains within its digits the instructions for its own transformation.

1.2 The Universal Growth Property

Core Claim: For any computable number $M \geq 2$:

$$\boxed{\text{DMFA}(M, 1) > M} \tag{2}$$

No fixed computable bound can withstand the DMFA transformation. Given any number — whether 3, 10^{100} , $TREE(3)$, or Rayo's Number — DMFA produces something strictly larger.

2 The Decoding Mechanism: From X to Parameters

2.1 The Philosophy of Structural Decoding

Traditional algorithms use **external parameters** — fixed constants chosen by the programmer. DMFA uses **internal parameters** — values extracted from the structure of the input number itself.

2.2 First Factory: The Four-Parameter Decoding

Definition 2.1 (First Factory Decoding). *The first factory receives $X = M^M$ and decodes it into **four parameters**:*

$$v = X + 1 \quad (\text{The Operand — the number to be operated on}) \quad (3)$$

$$op = X \bmod 3 \quad (\text{Operation Level: } 0=\text{addition}, 1=\text{multiplication}, 2=\text{exponentiation}) \quad (4)$$

$$c = \left\lfloor \frac{X}{2} \right\rfloor + 1 \quad (\text{Iteration Count — how many times to apply the operation}) \quad (5)$$

$$F = (X \bmod 5) + 2 \quad (\text{Pipeline Width: number of factories, } \in \{2, 3, 4, 5, 6\}) \quad (6)$$

Param	Name	Formula	Purpose
v	Operand	$v = X + 1$	The number that will be transformed by the hyperoperation. Always greater than X , ensuring growth.
op	Operation Level	$op = X \bmod 3$	Determines which hyperoperation to apply: addition (0), multiplication (1), or exponentiation (2).
c	Iteration Count	$c = \lfloor X/2 \rfloor + 1$	The repetition count for the hyperoperation. Large X yields large c , amplifying growth.
F	Pipeline Width	$F = (X \bmod 5) + 2$	Only in Factory 1: determines how many factories will run in this cycle (range: 2–6).

Table 1: The four parameters decoded from $X = M^M$ in the first factory

2.3 Why These Four Parameters?

Each parameter serves a distinct and critical role:

- $v = X + 1$: Ensures the operand is always strictly larger than the input X , guaranteeing growth at the hyperoperation stage.
- $op = X \bmod 3$: Cycles through addition (0), multiplication (1), and exponentiation (2), creating varied operational behavior based on the input's mathematical properties.
- $c = \lfloor X/2 \rfloor + 1$: Provides a large iteration count proportional to X . When X is enormous (e.g., $TREE(3)^{TREE(3)}$), c becomes equally enormous.
- $F = (X \bmod 5) + 2$: Restricts the pipeline width to a manageable range (2–6 factories per cycle), ensuring deterministic execution while still encoding cycle structure into the seed.

2.4 Subsequent Factories: The Three-Parameter Decoding

Definition 2.2 (Subsequent Factory Decoding). *Factories $m \geq 2$ receive $X_m = R_m^{R_m}$ from the previous factory and decode **only three parameters**:*

$$v_{m+1} = X_m + 1 \quad (7)$$

$$op_{m+1} = X_m \bmod 3 \quad (8)$$

$$c_{m+1} = \left\lfloor \frac{X_m}{2} \right\rfloor + 1 \quad (9)$$

Critical Rule: The pipeline width F is **not** re-decoded in subsequent factories. All factories $m \geq 2$ inherit the same F from Factory 1.

Why Only Factory 1 Determines F : The pipeline width must remain constant throughout a single cycle to ensure deterministic execution. If each factory could change F , the number of factories would fluctuate unpredictably, potentially creating infinite loops or premature termination. By fixing F at the first factory, we guarantee exactly F factories per cycle.

2.5 The Decoding Process: Complete Visual Diagram

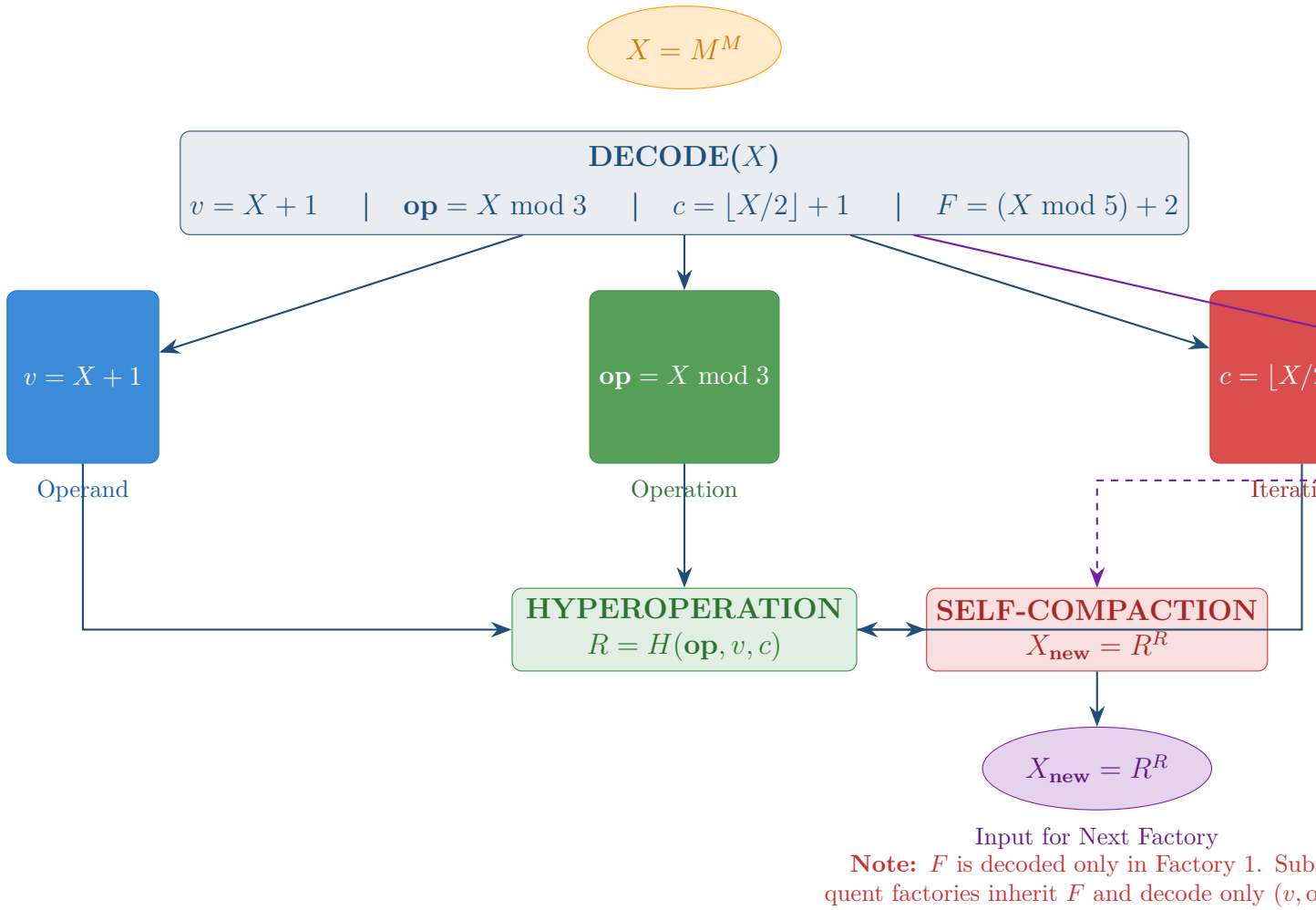


Figure 1: The complete decoding and transformation pipeline within a single factory. The input X is decoded into four parameters (three for subsequent factories), which drive the hyperoperation and self-compaction to produce the next factory's input.

3 The Factory Pipeline: A Worked Example

3.1 Example: DMFA(3, 1) — Step by Step

To illustrate the decoding mechanism concretely, we trace through a complete cycle with seed $M = 3$.

3.1.1 Phase 1: Initial Self-Exponentiation

$$X = M^M = 3^3 = 27 \tag{10}$$

The seed 3 is raised to itself, producing 27. This number 27 will now be decoded into operational parameters.

3.1.2 Factory 1: The Four-Parameter Decoding

Applying Definition 2.1 to $X = 27$:

$$\boxed{v} = 27 + 1 = \mathbf{28} \quad (11)$$

$$\boxed{\text{op}} = 27 \bmod 3 = \mathbf{0} \quad (\text{Addition}) \quad (12)$$

$$\boxed{c} = \lfloor 27/2 \rfloor + 1 = 13 + 1 = \mathbf{14} \quad (13)$$

$$\boxed{F} = (27 \bmod 5) + 2 = 2 + 2 = \mathbf{4} \quad (4 \text{ factories}) \quad (14)$$

Interpretation: Factory 1 will perform addition ($\text{op} = 0$) of the number 28 (v) repeated 14 times (c). The entire cycle will run 4 factories (F).

3.1.3 Factory 1: Hyperoperation and Self-Compaction

$$R_1 = H(0, 28, 14) = 28 + 14 = 42 \quad (15)$$

$$X_1 = R_1^{R_1} = 42^{42} \approx \mathbf{1.7 \times 10^{68}} \quad (69 \text{ digits}) \quad (16)$$

The result 42 is raised to itself, producing a 69-digit number that becomes the input for Factory 2.

3.1.4 Factory 2: Three-Parameter Decoding

Now $X_1 = 42^{42}$ is decoded using Definition 2.2:

$$v_2 = 42^{42} + 1 \approx \mathbf{10^{68}} \quad (17)$$

$$\text{op}_2 = (42^{42}) \bmod 3 = \mathbf{0} \quad (\text{Addition, since } 42 \equiv 0 \pmod{3}) \quad (18)$$

$$c_2 = \lfloor 42^{42}/2 \rfloor + 1 \approx \mathbf{5 \times 10^{67}} \quad (19)$$

Note: F is **not** decoded here. Factory 2 inherits $F = 4$ from Factory 1.

3.1.5 Factory 2: Hyperoperation and Self-Compaction

$$R_2 = H(0, 10^{68}, 5 \times 10^{67}) \approx \mathbf{1.5 \times 10^{68}} \quad (20)$$

$$X_2 = R_2^{R_2} \approx \mathbf{10^{10^{70}}} \quad (21)$$

The output now has approximately 10^{70} digits — a tower of height 2.

3.1.6 Factories 3 and 4

The pattern continues:

- **Factory 3:** Decodes $X_2 \approx 10^{10^{70}}$, producing an output with approximately $10^{10^{10^{70}}}$ digits (tower of height 3)
- **Factory 4:** Decodes X_3 , producing an output with tower height 4

3.1.7 Phase 4: The Grand Reset

After Factory 4 completes, the final operand v_4 (already a tower of height 4) is compacted:

$$\Phi = ((v_4)^{v_4})^F = ((v_4)^{v_4})^4 \quad (22)$$

This becomes the output of DMFA(3,1) and the seed for the next cycle.

3.2 Complete DMFA Cycle: Visual Diagram

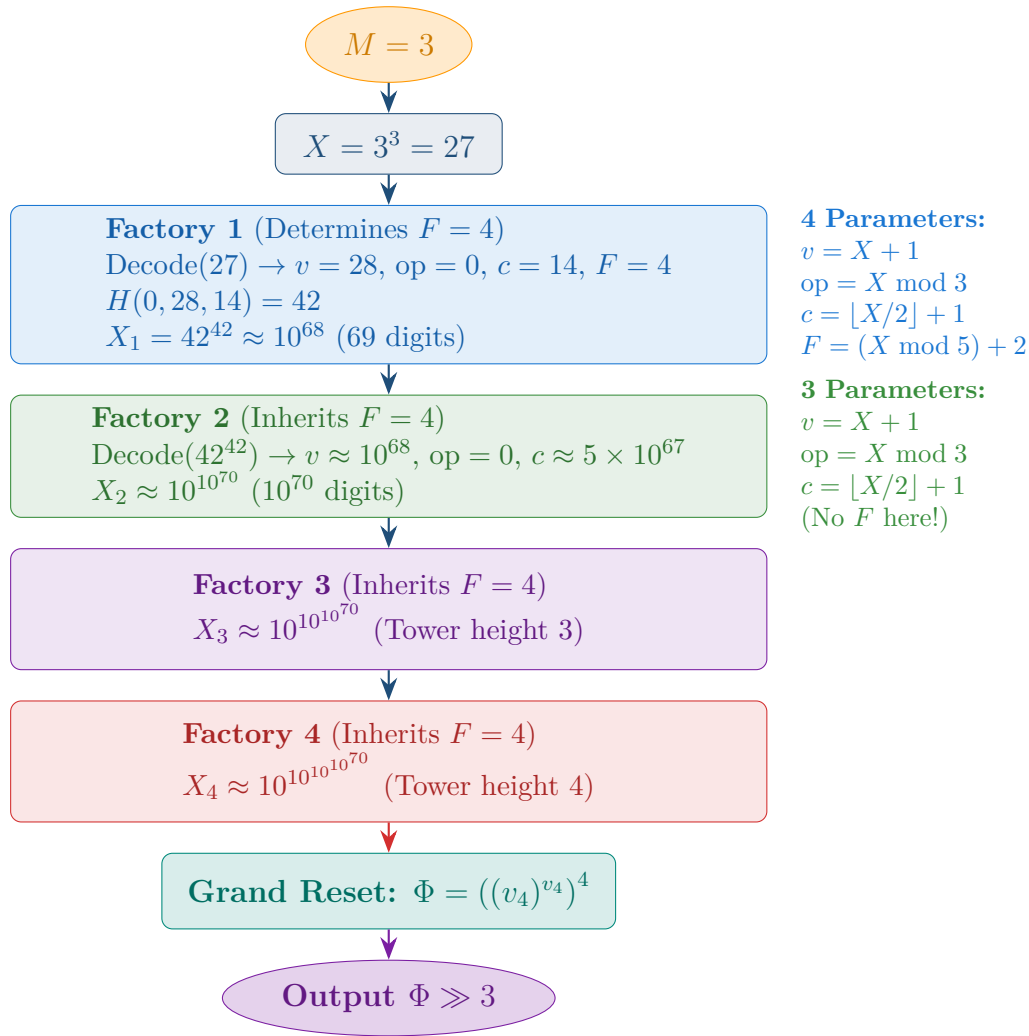


Figure 2: Complete trace of DMFA(3,1) showing the four-parameter decoding in Factory 1 and three-parameter decoding in subsequent factories. The pipeline width $F = 4$ is determined once and inherited throughout the cycle.

4 The Grand Reset: Encoding the Future

4.1 From Factory Output to New Seed

After the final factory F completes, the system performs a **Grand Reset** that compacts the cycle's output into a new seed:

$$\Phi = ((v_F)^{v_F})^F \quad (23)$$

This formula serves **two critical purposes**:

1. **Magnitude Preservation:** The double exponentiation $(v_F)^{v_F}$ ensures the output is vastly larger than any intermediate value.

2. **Cycle Memory:** Raising to the power F encodes the pipeline width into the new seed, influencing the next cycle's parameters.

4.2 How Φ Becomes the New Cycle's Seed AND Factory Count

This is the critical insight: The output Φ is not merely a large number. It serves **dual roles**:

4.2.1 Role 1: The New Seed

Φ becomes the input seed for the next cycle:

$$\text{DMFA}(\Phi, 1) \text{ — New cycle begins with seed } \Phi \quad (24)$$

When Φ is fed into the first factory of the next cycle, it undergoes self-exponentiation:

$$X_{\text{new}} = \Phi^\Phi \quad (25)$$

Then X_{new} is decoded into the four parameters for the new cycle's Factory 1:

$$v_{\text{new}} = \Phi^\Phi + 1 \quad (26)$$

$$\text{op}_{\text{new}} = (\Phi^\Phi) \bmod 3 \quad (27)$$

$$c_{\text{new}} = \left\lfloor \frac{\Phi^\Phi}{2} \right\rfloor + 1 \quad (28)$$

$$F_{\text{new}} = (\Phi^\Phi \bmod 5) + 2 \quad (29)$$

Because $\Phi \gg M$, the new parameters are vastly larger than the previous cycle's, creating compounded growth.

4.2.2 Role 2: The New Factory Count

The value $F_{\text{new}} = (\Phi^\Phi \bmod 5) + 2$ determines **how many factories** will run in the next cycle. Since Φ is enormous, F_{new} is essentially random within $\{2, 3, 4, 5, 6\}$, but it is deterministically derived from Φ itself.

goldaccent!20 Property	Determined By	Role
New Seed	Φ itself	Becomes M for next cycle
New Factory Count	$F_{\text{new}} = (\Phi^\Phi \bmod 5) + 2$	Determines cycle length (2–6 factories)
New Parameters	Decoded from Φ^Φ	Drive next cycle's operations

Table 2: The dual role of Φ in the Grand Reset

4.3 The Grand Reset Formula: Visual Breakdown

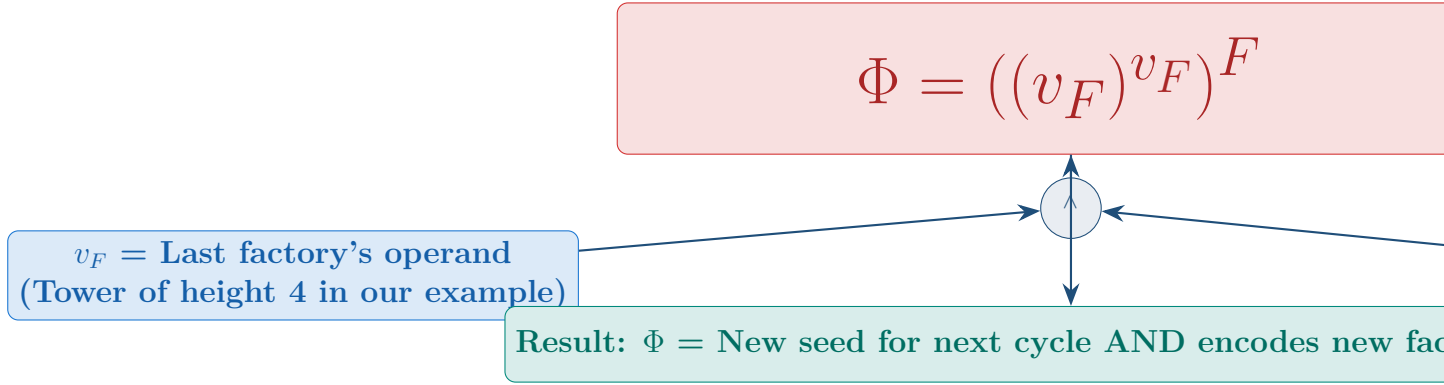


Figure 3: Visual breakdown of the Grand Reset formula $\Phi = ((v_F)^{v_F})^F$

4.4 The New Cycle Begins — Visual Feedback Loop

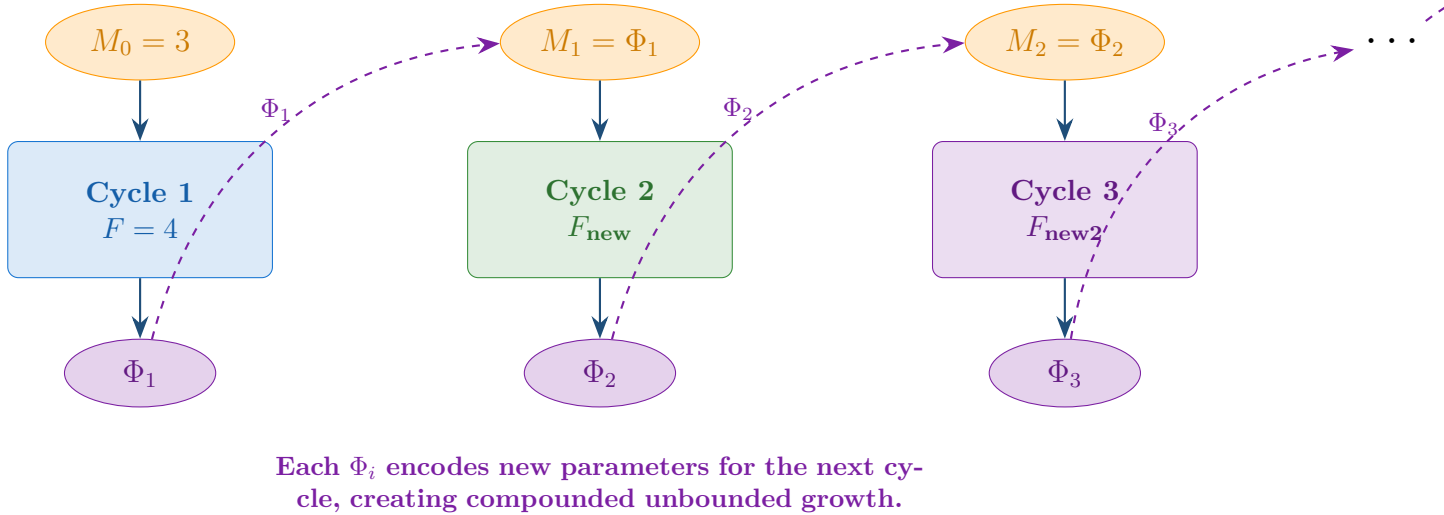


Figure 4: The multi-cycle feedback loop. Each cycle's output becomes the next cycle's seed, with the Grand Reset encoding both the magnitude and the pipeline width into the new starting value.

5 Theoretical Proofs

5.1 The Universal Growth Theorem

Theorem 5.1 (Universal Growth). *For any natural number $M \geq 2$:*

$$\boxed{DMFA(M, 1) > M} \quad (30)$$

Proof. Let $M \geq 2$ be arbitrary.

Phase 1: $X = M^M$. Since $M \geq 2$:

$$M^M \geq M^2 > M \quad (31)$$

Factory 1: The decoded operand is:

$$v = X + 1 = M^M + 1 > M^M > M \quad (32)$$

Hyperoperation: For any $\text{op} \in \{0, 1, 2\}$:

- If $\text{op} = 0$: $H(0, v, c) = v + c > v > M$
- If $\text{op} = 1$: $H(1, v, c) = v \cdot c > v > M$
- If $\text{op} = 2$: $H(2, v, c) = v^c > v > M$

Thus $R = H(\text{op}, v, c) > M$.

Self-Compaction:

$$R^R > R > M \quad (33)$$

Grand Reset: Since $F \geq 2$ and $v_F \geq R$:

$$\Phi = ((v_F)^{v_F})^F \geq (R^R)^2 > R^R > M \quad (34)$$

Therefore $\text{DMFA}(M, 1) = \Phi > M$. ■

□

5.2 Monotonicity and Divergence

Theorem 5.2 (Strict Monotonicity). *For all $M \geq 2$ and all $k \geq 0$:*

$$\boxed{\text{DMFA}(M, k+1) > \text{DMFA}(M, k)} \quad (35)$$

Proof. By definition:

$$\text{DMFA}(M, k+1) = \text{DMFA}(\text{DMFA}(M, k), 1) \quad (36)$$

Let $N = \text{DMFA}(M, k)$. By Theorem 5.1:

$$\text{DMFA}(N, 1) > N \quad (37)$$

Therefore $\text{DMFA}(M, k+1) > \text{DMFA}(M, k)$. ■

□

Theorem 5.3 (Unbounded Divergence). *For all $M \geq 2$:*

$$\boxed{\lim_{k \rightarrow \infty} \text{DMFA}(M, k) = \infty} \quad (38)$$

Proof. By Theorem 5.2, the sequence is strictly increasing. If bounded by B , then $\text{DMFA}(B, 1) > B$ by Theorem 5.1, contradiction. ■

□

6 Position in the Fast-Growing Hierarchy

maincolor!15 Level	FGH Ordinal	Examples	Computability
Primitive Recursive	$< \omega$	Addition, Multiplication, Exponentiation	Computable
Ackermann Level	ω	Ackermann $A(n, n)$	Computable
goldaccent!15 DMFA Level	$\omega + 1$	DMFA(n, k) for fixed k	Computable
Iterated DMFA	$\omega + 2$	DMFA(n, n)	Computable
Proof-Theoretic	Γ_0	$TREE(3)$, $SSCG(3)$	Computable
accentred!10 Busy Beaver	Beyond computable	BB(n)	Non-computable

Table 3: Position of DMFA in the Fast-Growing Hierarchy

7 Comparison with Classical Large Numbers

maincolor!15 Property	Graham’s Number	TREE(3)	Rayo’s Number	DMFA
Type	Fixed constant	Fixed constant	Fixed constant	Growth function
Can grow?	No	No	No	Yes — unbounded
Needs input?	No	No	No	Yes (any $M \geq 2$)
Output	Single number	Single number	Single number	Always larger than i
Iteratable?	N/A	N/A	N/A	Infinite cycles
FGH Ordinal	$\omega + 1$	Γ_0	$> \Gamma_0$	$\omega + 1$

Table 4: DMFA vs. classical large numbers: the fundamental difference

8 Growth Analysis

8.1 Growth Trajectory for Seed $M = 3$

maincolor!15 Stage	Approximate Size	Description
Input	3	Single digit
After Factory 1	$\approx 10^{68}$	69-digit number
After Factory 2	$\approx 10^{10^{70}}$	Tower of height 2
After Factory 3	$\approx 10^{10^{10^{70}}}$	Tower of height 3
After Factory 4	$\approx 10^{10^{10^{10^{70}}}}$	Tower of height 4
After Grand Reset	$((v_4)^{v_4})^4$	Tower of height 4, raised to 4th power
After Cycle 2	Unfathomably larger	Parameters decoded from Φ^Φ

Table 5: DMFA growth trajectory starting from $M = 3$

8.2 The Compounding Effect

Each cycle of DMFA produces output Φ that is:

1. **Self-exponentiated** (Φ^Φ) to create the next cycle's X
2. **Modulo-decoded** to determine the next cycle's operations
3. **Used as seed** for the next iteration

This creates a **triple compounding**:

- **Magnitude compounding:** Each Φ is vastly larger than its predecessor
- **Operation compounding:** Larger X means larger c (more iterations)
- **Structural compounding:** The factory count F can vary, creating unpredictable but deterministic growth patterns

9 Complete Algorithm: Pseudocode

Algorithm: DMFA(M, k) — **Complete Implementation**

Input: Seed $M \geq 2$, Number of cycles $k \geq 1$

Output: Final value after k cycles

```
function DMFA(M, k):
    current_seed = M

    for cycle = 1 to k:
```

```

// Phase 1: Self-exponentiation
X = current_seed ^ current_seed

// Factory 1: Four-parameter decoding
v = X + 1
op = X mod 3
c = floor(X / 2) + 1
F = (X mod 5) + 2    // Pipeline width (2 to 6)

// Factory 1 execution
R = Hyperoperation(op, v, c)
X = R ^ R

// Factories 2 to F: Three-parameter decoding
for factory = 2 to F:
    v = X + 1
    op = X mod 3
    c = floor(X / 2) + 1

    R = Hyperoperation(op, v, c)
    X = R ^ R
end for

// Phase 4: Grand Reset
// v_F is the final operand from Factory F
Phi = ((v_F) ^ (v_F)) ^ F

// Phi becomes the seed for next cycle
// AND determines the next cycle's factory count
current_seed = Phi
end for

return current_seed
end function

```

10 Conclusion

The Dynamic Meta-Factory Automaton represents a fundamental shift from constructing large numbers to constructing **processes that grow numbers**. Its key innovation — **structural self-parameterization** — allows any input to be decoded into its own operational parameters, creating a closed feedback loop of unbounded growth.

Unlike static constants like $TREE(3)$ or Graham's Number, DMFA is:

- **Universal:** Works on any input $M \geq 2$
- **Deterministic:** Always produces the same output for the same input
- **Unbounded:** Iteration yields infinite growth
- **Computable:** Explicit algorithm with no oracle requirements

“The largest number is not a number at all.
It is a process that makes numbers larger.”

References

- [1] Graham, R.L. *Concrete Mathematics*. Addison-Wesley, 1994.
- [2] Friedman, H.M. “Finite Functions and Large Cardinals.” *Annals of Math*, 1998.
- [3] Knuth, D.E. *The Art of Computer Programming*. Addison-Wesley, 1997.
- [4] Rayo, A. “The Largest Number Ever.” MIT, 2007.
- [5] Radó, T. “On Non-Computable Functions.” *BSTJ*, 1962.